

CYBERTRAINING 2026 • CAPSTONE PROJECT

ML Approximation of Hessian Components for Accelerated CASSCF Optimization

Toward Resonating Hartree-Fock

Qabas Mohammad Al Khatib (PhD candidate)

Department of Chemistry, Case Western Reserve University

Advisor: Dr. Shane M. Parker

Why Hessian Information Matters

- 1 Resonating Hartree-Fock (ResHF) is designed for strongly correlated systems using nonorthogonal Slater determinants.
- 2 Its optimization simultaneously adjusts both orbital coefficients and CI weights — a fully coupled, nonlinear problem that often fails to converge.
- 3 Second-order methods (like Newton-Raphson) use the Hessian for reliable convergence, but computing the exact Hessian is very expensive.
- 4 This project asks a practical question: Can machine learning approximate key Hessian components from much cheaper features, starting with CASSCF as a testbed?

Introducing Resonating Hartree-Fock (ResHF)

- **ResHF Wave Function:** A linear combination of nonorthogonal Slater determinants

$$\left| \Psi_I^{\text{ResHF}} \right\rangle = c_{IA} \left| \Phi_A \right\rangle + c_{IB} \left| \Phi_B \right\rangle + c_{IC} \left| \Phi_C \right\rangle + \dots, \text{ where } \left\langle \Phi_A \left| \Phi_B \right\rangle \neq 0$$

- With **nonorthogonal sets** of molecular orbitals:

$$\left| \Phi_A \right\rangle = \left| \chi_1^A \dots \chi_i^A \right\rangle, \text{ where } \left\langle \chi_i^A \left| \chi_j^A \right\rangle = 0$$

$$\left| \Phi_B \right\rangle = \left| \chi_1^B \dots \chi_i^B \right\rangle, \text{ where } \left\langle \chi_i^B \left| \chi_j^B \right\rangle = 0$$

$$S_{ij}^{AB} = \left\langle \chi_i^A \left| \chi_j^B \right\rangle \neq 0$$

- Optimization involves both:

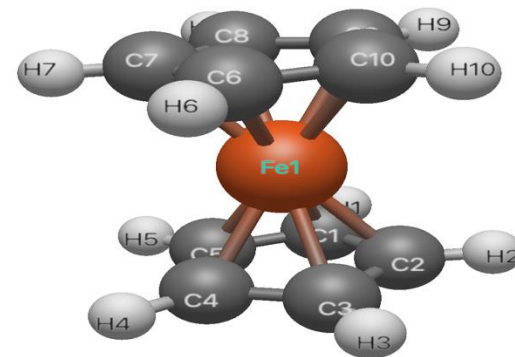
$$\text{molecular orbitals: } \{ \chi_i^A, \chi_j^B, \dots \}$$

$$\text{state expansion coefficients: } \{ c_{IA}, c_{IB}, \dots \}$$

MOTIVATION

The Hessian Cost Problem

Example: ferrocene, SCF-NR/ def2-SVP



Basis functions

$$n = 221$$

Electrons

$$96 \rightarrow \text{occupied orbitals } n_{\text{occ}} = 48$$

Virtual orbitals

$$n_{\text{vir}} = n - n_{\text{occ}} = 221 - 48 = 173$$

rotation-space dimension

$$n_{\text{occ}} \times n_{\text{vir}} = 48 \times 173 = 8,304$$

Full Hessian size

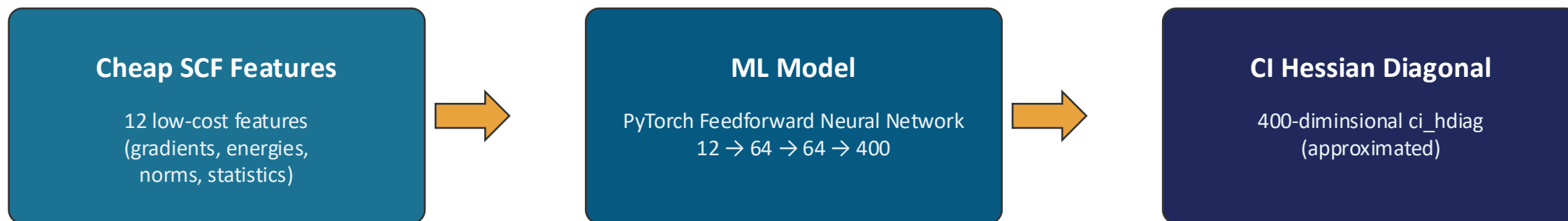
$$8,304 \times 8,304 \approx 69 \text{ million entries}$$

Full Hessian size: ~69 million entries; rebuilt (or inverted) at every optimization step.

This is why exact second-order methods become impractical for larger systems.

The Core Idea

Can cheap SCF-level features predict expensive Hessian information?



$$\hat{h}_{cc} = f_{\theta}(x_{\text{SCF}}) \approx \text{diag}(H_{cc})$$

Hypothesis: cheap SCF-level descriptors contain enough information to approximate diagonal Hessian curvature

FORMULATION

Second-order optimization solves the coupled Newton-Raphson update:

$$\begin{pmatrix} \kappa \\ \mathbf{c} \end{pmatrix}_{n+1} = \begin{pmatrix} \kappa \\ \mathbf{c} \end{pmatrix}_n - \mathbf{H}^{-1}(\kappa_n, \mathbf{c}_n) \mathbf{g}(\kappa_n, \mathbf{c}_n)$$

$\mathbf{H}$ is the full Hessian and $\mathbf{g}$ is the gradient.

Full Coupled Hessian

$$\mathbf{H} = \begin{pmatrix} H_{\kappa\kappa} & H_{\kappa c} \\ H_{c\kappa} & H_{cc} \end{pmatrix}$$

κ =orbital rotation parameters, c = CI coefficients

Our Capstone's ML Target

- We focus on the **CI Hessian diagonal block** (H_{cc}), specifically the vector `ci_hdiag`.
- This part is readily available from PySCF.
- **Not addressed in this project:** The full orbital-CI coupling block ($H_{\kappa c}$).
- This is a genuine limitation of the current PySCF interface, not an oversight.

Data: CASSCF Data Generation

INVEST Molecule Set: Pollice et al., Matter 4, 1654–1682 (2021)

20 / 23

molecules converged (of staged)

(6,6)

active space, def2-SVP basis

12

SCF-level input features

Held-Out Test Molecules (6)

S1000

test molecule 1

S1200

test molecule 2

S1180

test molecule 3

S1010

test molecule 4

S1050

test molecule 5

S1090

test molecule 6

14 additional molecules used for training, drawn from the same INVEST benchmark set

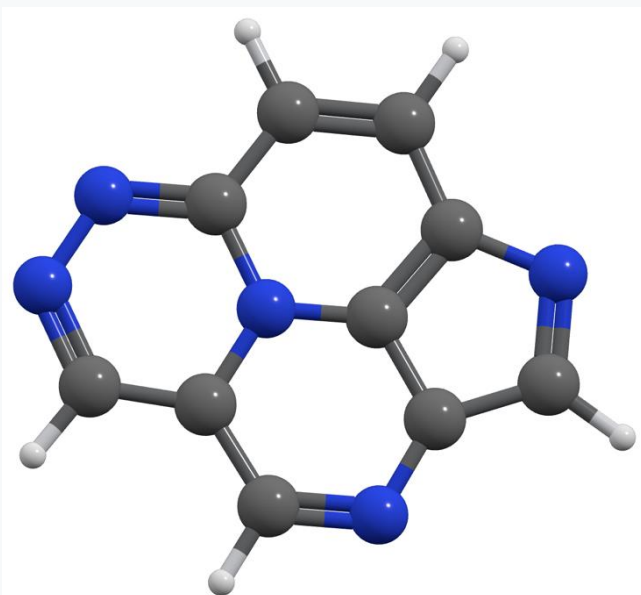


train/test split — verified

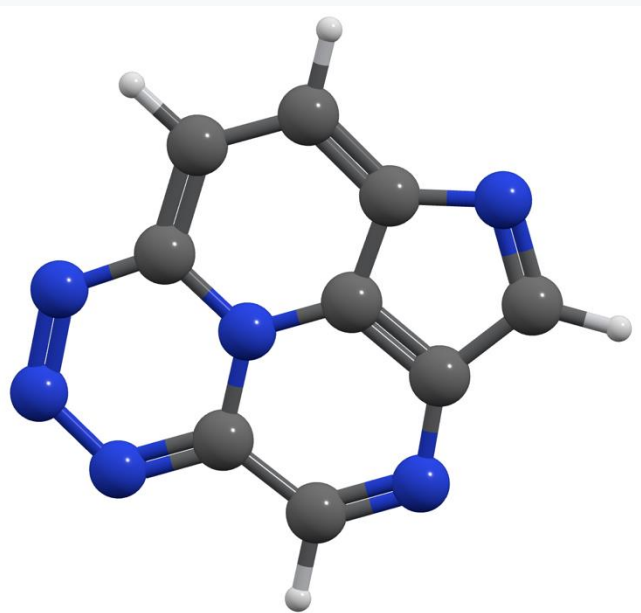
```
X_train, X_test, y_train, y_test, names_train, names_test = train_test_split(  
    X, y, np.array(names), test_size=6, random_state=42  
)
```

Sample INVEST Molecule Structures

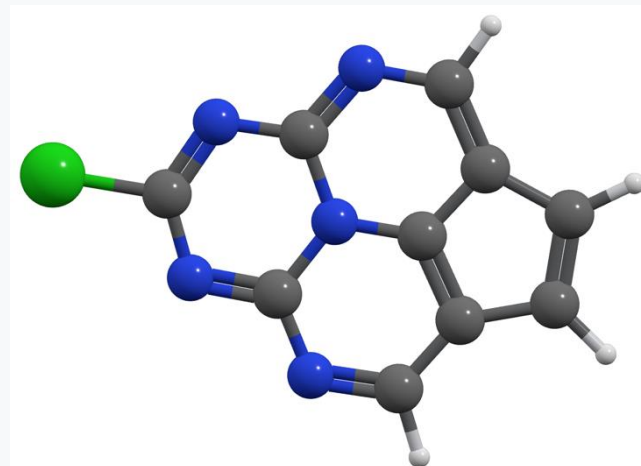
Representative structures from the nitrogen-rich fused-ring INVEST scaffold set



S1000



S1010



S1120

Nitrogen-rich, fused polycyclic frameworks with varying substitution patterns — consistent with the structural diversity of the Pollice et al. INVEST benchmark set.

Feature Engineering

Because orbital space sizes vary across molecules, we summarize the data into 12 fixed-size features per molecule:

- Gradient statistics (g_orb)
- CI coefficient statistics
- Total energy
- Hessian-diagonal statistics
- Vector norms
- Active-space size descriptors

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

feature_engineering — verified

```
scaler_X = StandardScaler(); X_train_scaled = scaler_X.fit_transform(X_train)
scaler_y = StandardScaler(); y_train_scaled = scaler_y.fit_transform(y_train)
```

14

training molecules

6

held-out test

This creates a consistent input for the neural network.

Two Models Compared

Note: These solve different prediction tasks, not a direct architecture comparison.

PyTorch FFNN — Full Target

12 → 64 → 64 → 400

- Predicts the full 400-dimensional ci_hdiag vector
 - ReLU hidden layers · MSE loss

ChemML MLP — Baseline Proxy

12 → 64 → 64 → 1

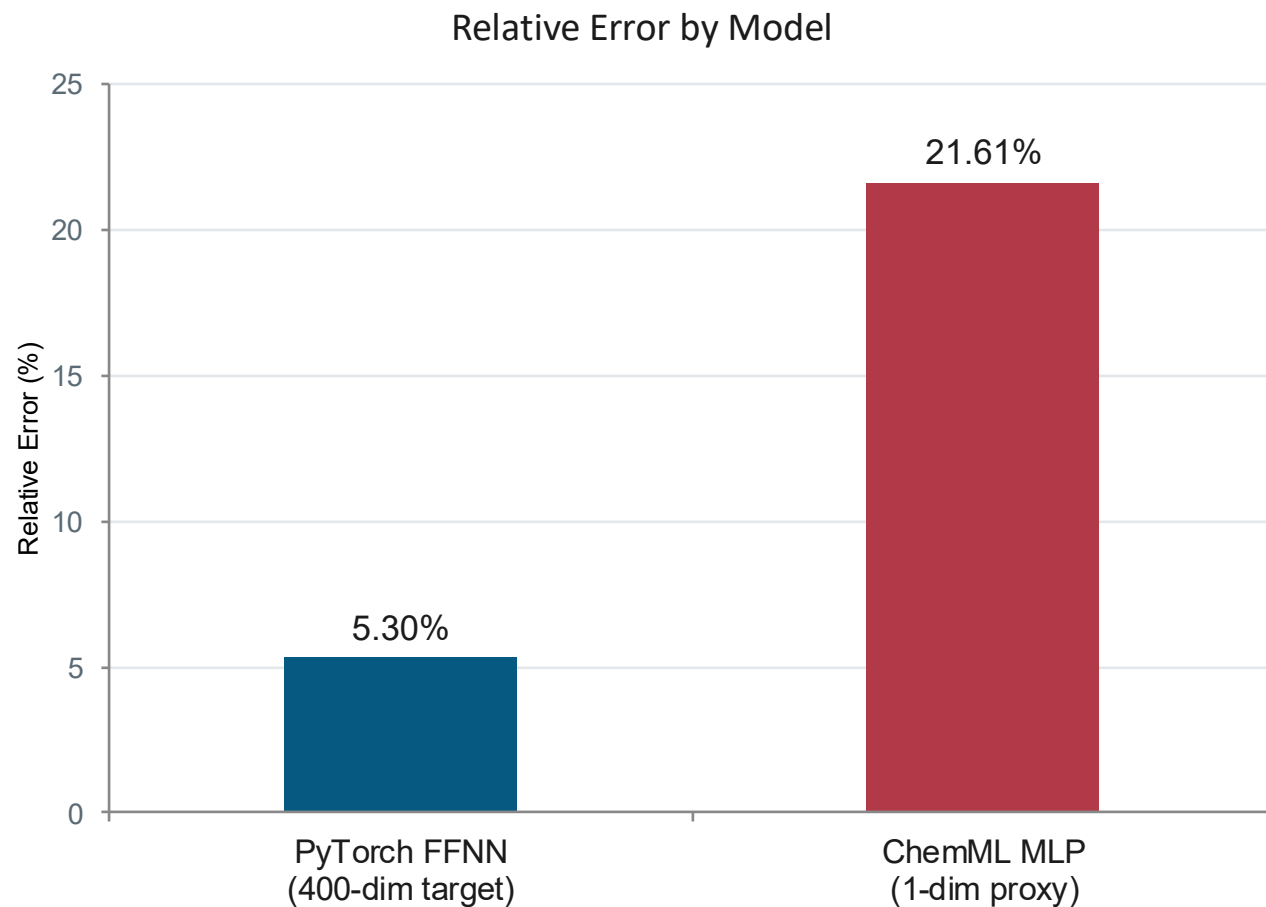
- Single-output regression by design
- predicts only the mean of ci_hdiag

chemml baseline — verified notebook code

```
from chemml.models import MLP

model_chemml = MLP(
    engine='pytorch', nfeatures=12, nneurons=[64, 64, 1],
    activations=['ReLU', 'ReLU', 'linear'], learning_rate=0.001,
    nepochs=2000, batch_size=14, loss='mean_squared_error',
    is_regression=True, opt_config='adam'
```

Model Comparison



- **PyTorch** achieves significantly lower error (5.30%) compared to the **ChemML** baseline (21.61%).
- Note: The models solve different prediction tasks (see Methods).

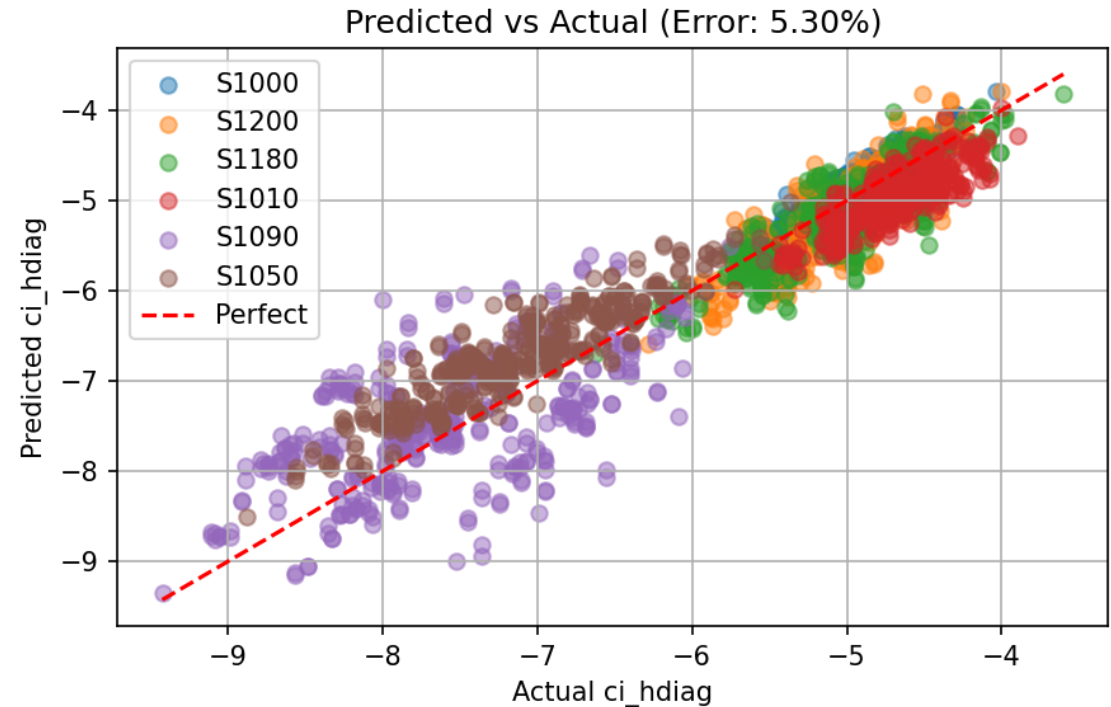
```
S1000: Actual=0.9426, Pred=1.1350
      Error=20.41%
S1200: Actual=0.8295, Pred=1.0811
      Error=30.33%
S1180: Actual=0.8610, Pred=0.8682
      Error=0.83%
S1010: Actual=1.2144, Pred=0.4546
      Error=62.56%
S1050: Actual=-1.3183, Pred=-1.1160
      Error=15.35%
S1090: Actual=-1.7744, Pred=-1.8609
      Error=4.87%

PyTorch overall: 5.30%
ChemML overall: 21.61%
```

RESULTS

Training Convergence

PyTorch FFNN — full run, notebook 02



plotting call — verified

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))  
axes[0].plot(losses); axes[0].set_yscale('log') # Training Loss
```

Honest Limitations

What this proof of concept does not yet claim

Small dataset

20 of 23 staged molecules converged; only 6 held out for testing, no cross-validation across the full set yet.

Orbital-CI Coupling Gap

The orbital–CI coupling block remains inaccessible via PySCF's `gen_g_hop` and is not addressed by this pipeline.

CASSCF $\neq$ ResHF

Orthogonal-orbital **CASSCF is a proxy**, not a structural stand-in for ResHF's nonorthogonal formulation.

ChemML comparison caveat

The 21.61% ChemML figure reflects a 1-dim proxy target, not a fair capability ranking against PyTorch.

From CASSCF to ResHF

Features $\rightarrow$ ML $\rightarrow$ Hessian diagonal: a template for eventual ResHF-specific Hessian learning, once nonorthogonal orbital structure is incorporated.

Next Steps

- 1 Cross-validation across all 20 INVEST molecules (per-molecule error analysis)
- 2 Feature importance analysis on the 12 SCF-level descriptors
- 3 Extend feature representation toward nonorthogonal (ResHF) orbital structure
- 4 Integrate learned/approximate Hessian into second-order ResHF optimizer

Conclusion

- Demonstrated an ML pipeline predicting the CI Hessian diagonal from cheap SCF-level features on CASSCF(6,6)/def2-SVP.
- 5.30% relative error on held-out INVEST molecules for the full 400-dim target.
- A transparent, honestly-scoped proof of concept , with a clear path toward ResHF's second-order optimization.

Thank you. Questions?